

1、 高考录取统分子系统有如下功能：（本小题5分）

（1） 计算标准分：根据考生原始分计算，得到标准分，存入考生分数文件；

（2） 计算录取分数线：根据标准分、招生计划文件中的招生人数，计算录取线，
存储进录取线文件；

试根据要求画出该系统的**初始软件结构图**。

2、某电器集团公司下属的厂包括技术科、生产科等基层单位。现在想建立一个计算机辅助企业管理系统，其中：（本小题10分）

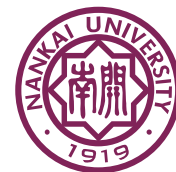
生产科的任务是：

- （1）根据销售公司转来的内部合同（产品型号、规格、数量、交获日期）制定车间月生产计划。
- （2）根据车间实际生产日报表、周报表调整日生产计划。
- （3）以月生产计划为依据，制定产品设计（结构、工艺）及产品组装月计划。
- （4）将产品的组装计划传达给各科，将组装月计划分解为周计划，下达给车间。

技术科的任务是：

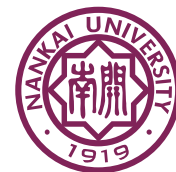
- （1）根据生产科转来的组装计划进行产品结构设计，产生产品装配图给生产科，产生外购需求计划给供应科，并产生产品自制物料清单。
- （2）根据组装计划进行产品工艺设计，根据产品自制物料清单产生工艺流程图给零件。

试根据要求画出该系统的**流程图**，并转为**初始软件结构图**。



南开大学
Nankai University

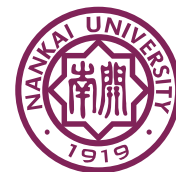
2.2 软件体系结构建模



模型的定义

模型：对无法直接观测的对象的可视化描述或近似

- 模型是对现实的简化
- 模型提供了系统的蓝图
- 模型可以是结构化的，着重描述系统的组织架构
- 模型也可以是行为化的，着重描述系统的动态行为



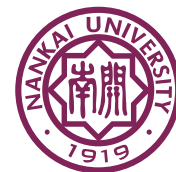
为什么建模？

建模可以让我们更好地理解我们构建的系统。

通过建模，实现了四个目标：

- 对系统进行可视化。
- 规约系统的结构或行为。
- 用于指导构建系统的模板。
- 将设计决策形成为文档。

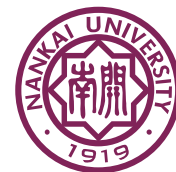
软件体系结构建模的种类



南开大学
Nankai University

- 结构模型
- 框架模型
- 动态模型
- 过程模型
- 功能模型

软件体系结构建模的种类



南开大学
Nankai University

结构模型

这是一个最直观、最普遍的建模方法。这种方法以体系结构的构件、连接件和其他概念来刻画结构，并力图通过结构来反映系统的重要语义内容，包括系统的配置、约束、隐含的假设条件、风格、性质等。

研究结构模型的核心是体系结构描述语言。

软件体系结构建模的种类



南开大学
Nankai University

框架模型

框架模型与结构模型类似，但它不太侧重描述结构的细节而更侧重于整体的结构。

框架模型主要以一些特殊的问题为目标建立只针对和适应该问题的结构。

软件体系结构建模的种类

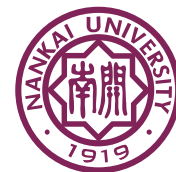


南开大学
Nankai University

动态模型

动态模型是对结构或框架模型的补充，研究系统的“大颗粒”的行为性质。例如，描述系统的重新配置或演化。动态可以指系统总体结构的配置、建立或拆除通信通道或计算的过程。

软件体系结构建模的种类



南开大学
Nankai University

过程模型

过程模型研究构造系统的步骤和过程。

因而结构是遵循某些过程脚本的结果。

软件体系结构建模的种类

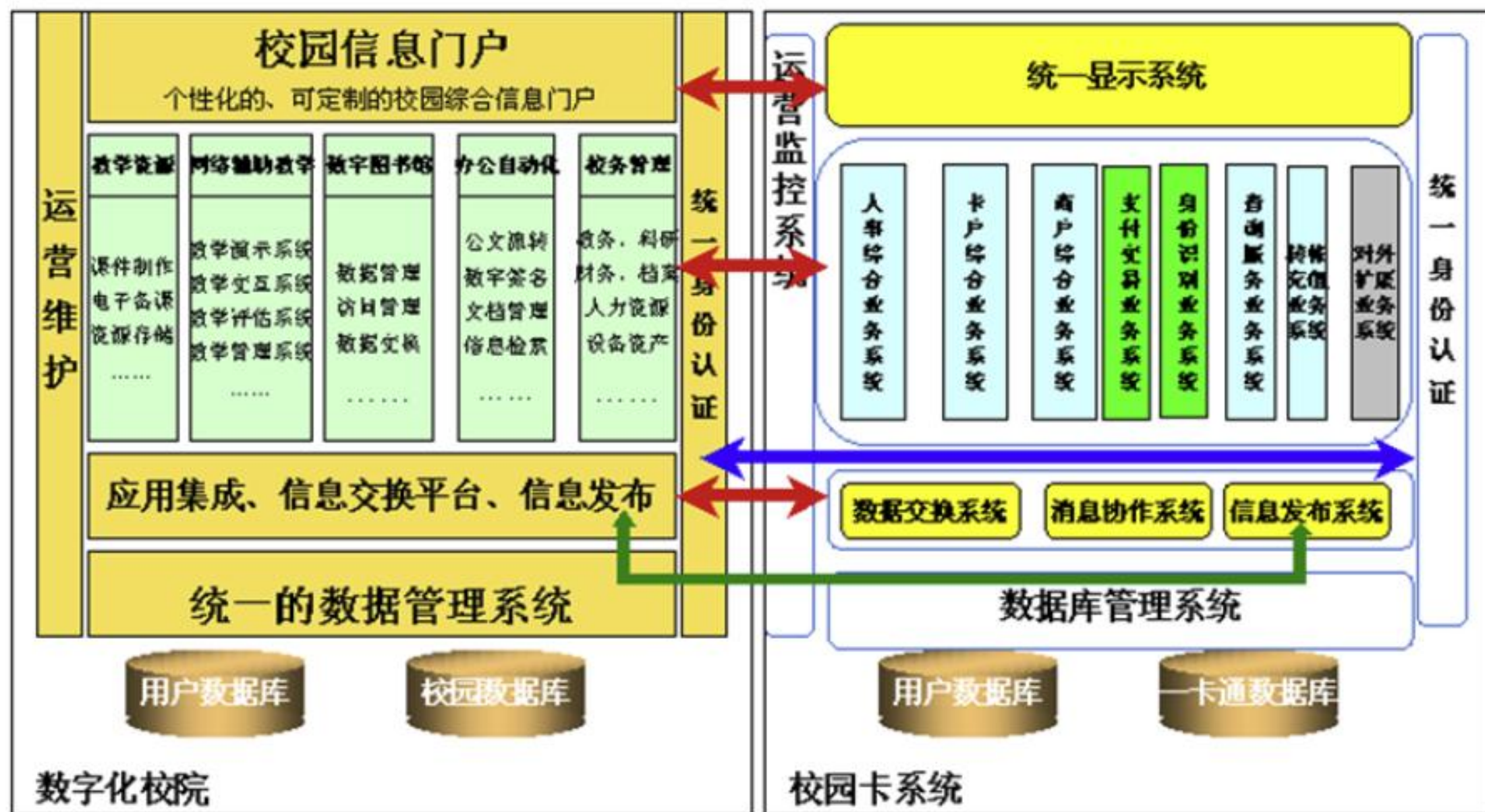


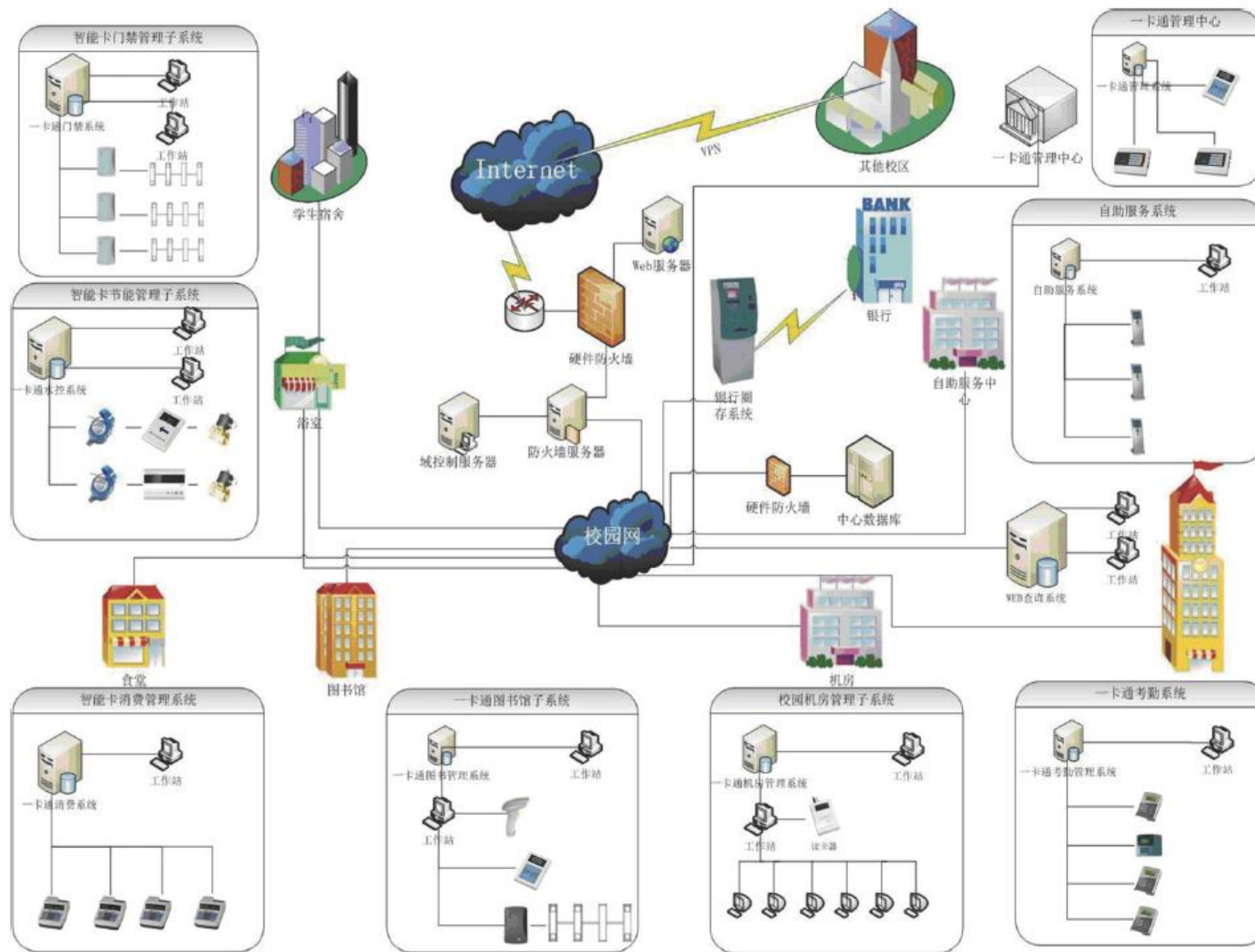
南开大学
Nankai University

功能模型

功能模型认为体系结构是由一组功能构件按层次组成，下层向上层提供服务。

功能模型可以看作是一种特殊的框架模型。





开发架构

设计开发

开发环境

团队建设

测试

测试、调试

性能、调优

交付

版本发布

版本管理

产品支持架构

配置管理

测试与产品
发布同步

应用软件架构

应用集成

应用分布

系统集成

界面交互

状态同步

系统恢复

企业应用集成

通讯标准

过程目录

业务逻辑

软件架构

通用业务组件

程序架构服务

数据架构

数据访问对象

数据架构服务

技术架构

执行架构

表示层 中间层 数据层 批处理

通信 (层间)

应用/数据集成 (继承/面向对象)

底层集成

数据集成

数据分布

远程数据存取

数据备份与同步

数据恢复

数据仓库

格式转换

数据存储

基础架构

用户设备集成

胖/瘦客户端

移动设备

点对点网关

基础服务

J2EE 产品
(如应用服务器)

硬件/系统软件
配置管理

服务器
集群

设备
恢复

基础平台

硬件与外
围设备

系统软件

网络存储

内部网络

网络集成

互联网集成

广域网集成

工业/商业
网络集成

运行架构

参数维护

引用数据维护

过程参数维护

性能管理

流量控制

动态资源分配

内部运行

检查架构

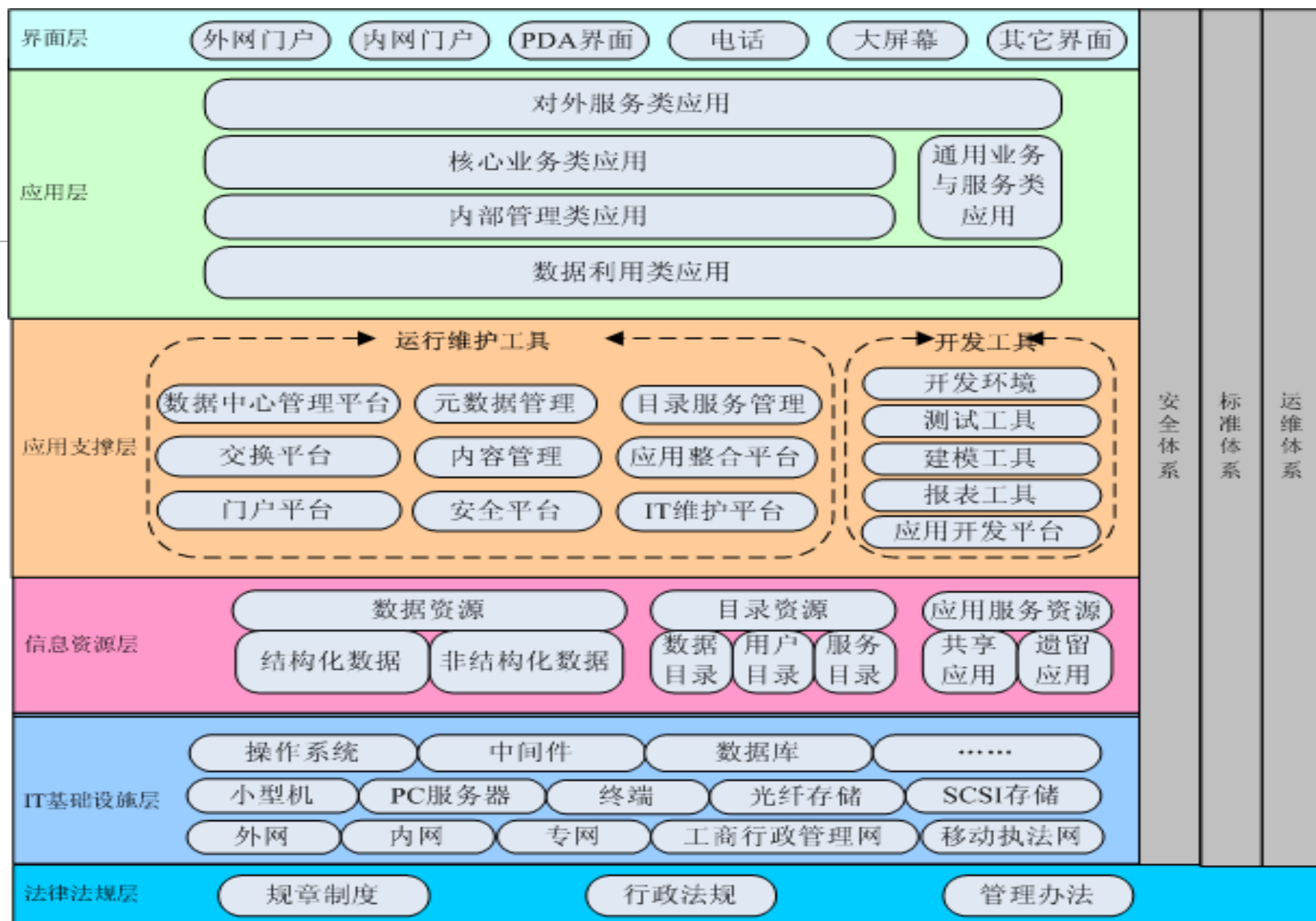
端对端监测

运行监视与
预防性恢复服务

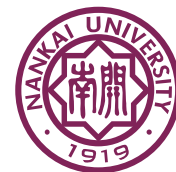
系统管理

应用/系统
可操作性

系统集中管理



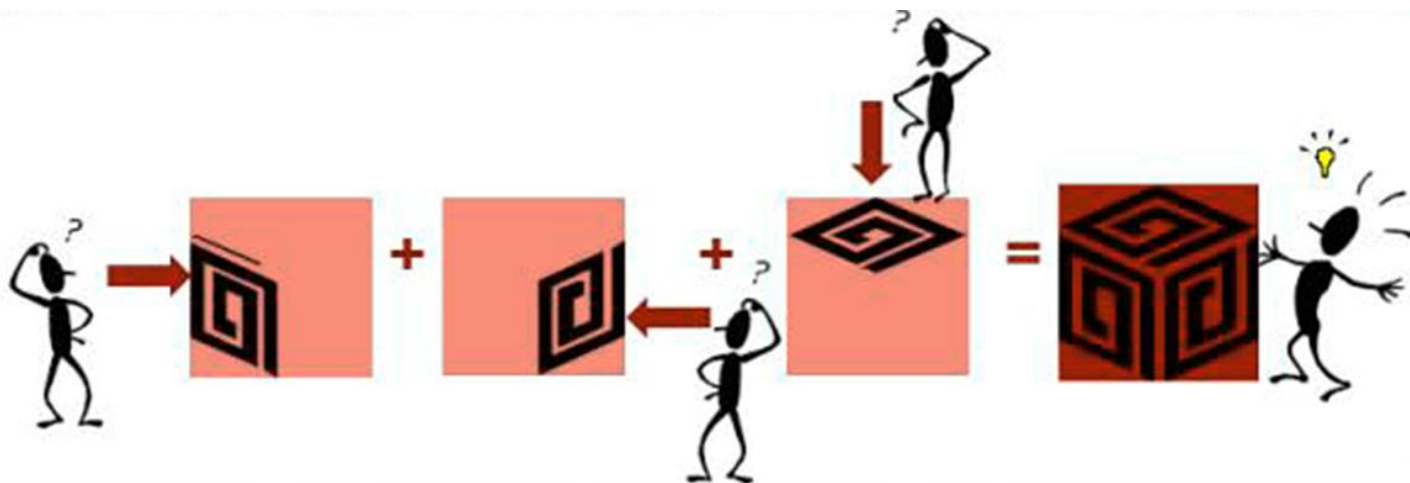
基于视图的体系结构建模



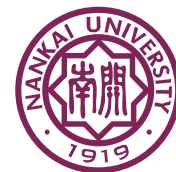
南开大学
Nankai University

“视图”的概念为我们提供了描述体系结构的原则：

- 记录相关的意见
- 然后添加适用于多个视图的信息
- 从而将观点联系在一起



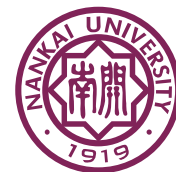
基于视图的体系结构建模



南开大学
Nankai University

- 架构元素-软件或硬件中存在的一组实际元素
- 视图-由系统相关者编写和读取的一组连贯的架构元素的表示
- 视图是一组架构元素及其关联关系的表示
- 但并非涵盖所有架构元素。
- 视图绑定了体系结构描述时感兴趣的元素类型和关系类型，并显示了它们。

基于视图的体系结构建模

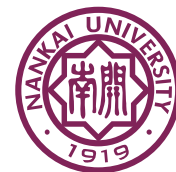


南开大学
Nankai University

视图是一种复杂的管理方法。

每个视图都可以回答有关体系结构的不同问题

- 主要执行单元和数据存储是什么？
- 其他软件如何使用该软件？
- 数据是如何在系统中流动的？
- 如何将软件部署到硬件上？



视图的类型

分解视图-显示与“关联的子模块”关系相关联的模块

使用视图-显示与“使用”关系相关联的模块（即，一个模块使用另一模块提供的服务）

分层视图-显示被划分为相关和连贯功能的模块组。每个组代表整体结构中的一个层。

类/泛化视图-显示称为类的模块，这些模块通过关系的“继承”或“实例”关联

视图的类型



南开大学
Nankai University

进程视图 - 显示通过通信、同步和排除操作连接的进程或线程

并发视图 - 显示组件和连接器，其中连接器代表“逻辑线程”

共享数据（存储库）视图 - 显示创建、存储和访问持久数据的组件和连接器

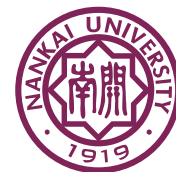
客户端-服务器视图 - 显示协作的客户端和服务端以及它们之间的连接器

部署视图 - 显示软件元素及其对硬件和通信元素的分配

实施视图 - 显示开发、集成和配置控制环境中的软件元素及其到文件结构的映射

工作分配视图 - 显示模块及其如何分配给负责实施和集成它们的开发团队

“4+1”的视图模型



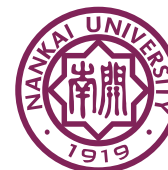
南开大学
Nankai University

Kruchten在1995年提出了“4+1”的视图模型。

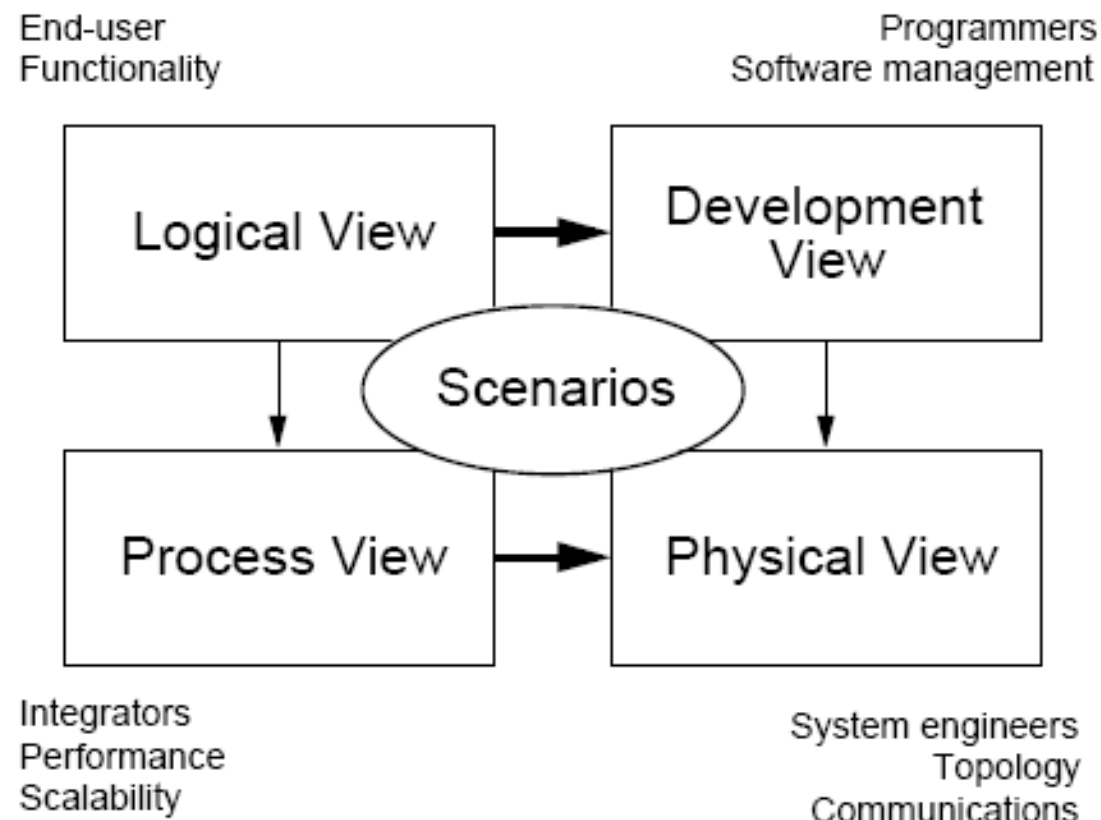
“4+1”视图模型从5个不同的视角包括逻辑视图、进程视图、物理视图、开发视图和场景视图来描述软件体系结构。

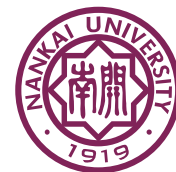
每一个视图只关心系统的一个侧面，5个视图结合在一起才能反映系统的软件体系结构的全部内容。

“4+1”的视图模型



南开大学
Nankai University

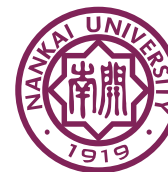




逻辑视图

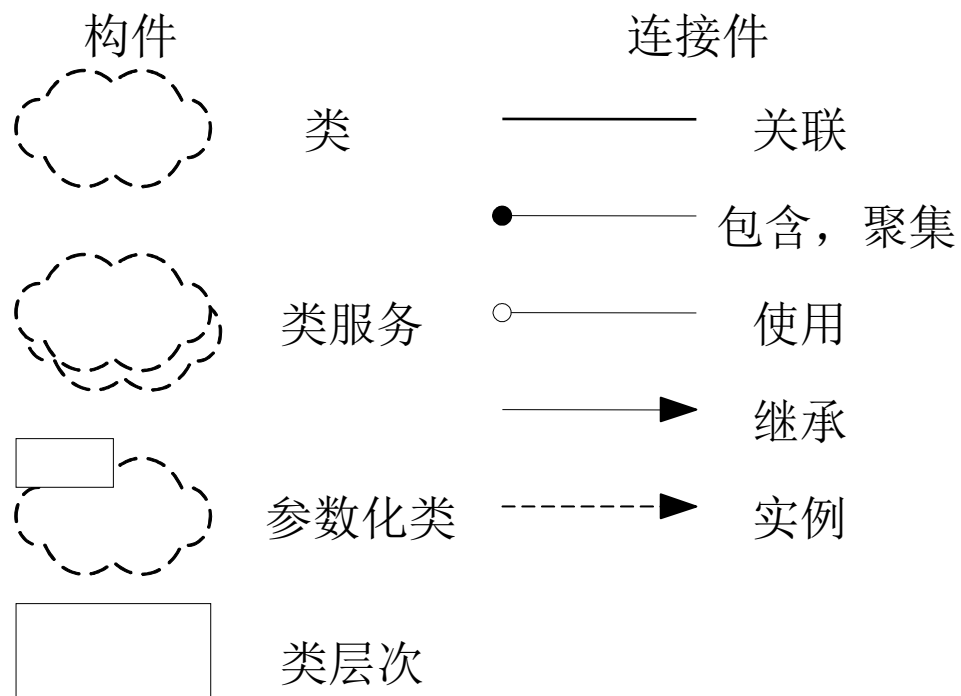
逻辑视图主要支持系统的功能需求，即系统提供给最终用户的服务。在逻辑视图中，系统分解成一系列的功能抽象，这些抽象主要来自问题领域。这种分解不但可以用来进行功能分析，而且可用作标识在整个系统的各个不同部分的通用机制和设计元素。

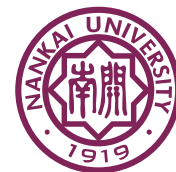
在面向对象技术中，通过抽象、封装和继承，可以用对象模型来代表逻辑视图，用类图来描述逻辑视图。



逻辑视图

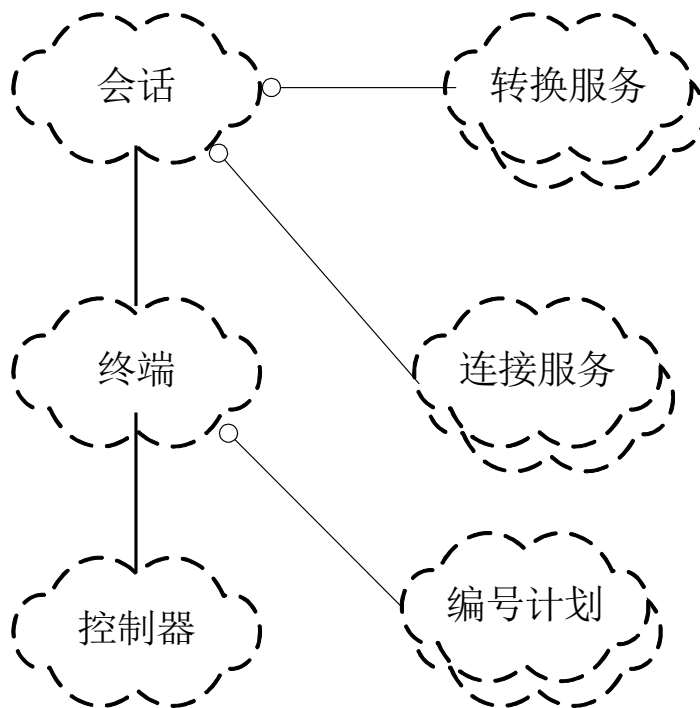
可以从Booch标记法中导出逻辑视图的标记法，只是从体系结构级的范畴来考虑这些符号，用Rational Rose进行体系结构设计。

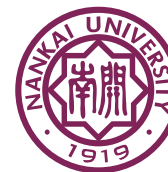




逻辑视图

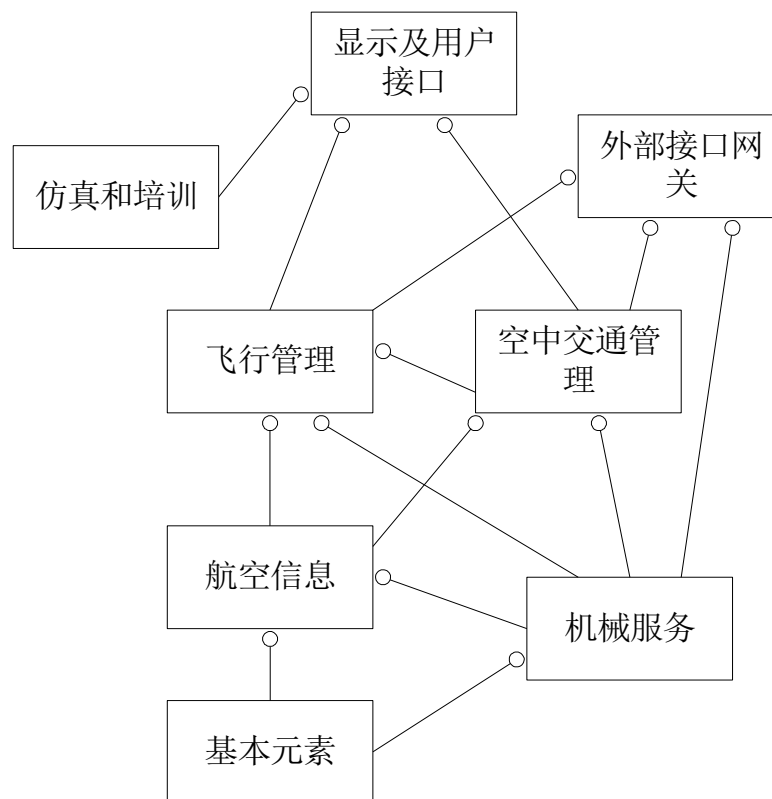
逻辑视图中使用的风格为面向对象的风格，逻辑视图设计中要注意的主要问题是要保持一个单一的、内聚的对象模型贯穿整个系统。



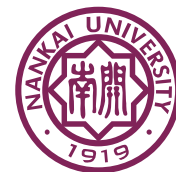


逻辑视图

对于规模更大的系统来说，体系结构级中包含数十甚至数百个类



开发视图



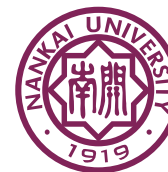
南开大学
Nankai University

开发视图也称模块视图，主要侧重于软件模块的组织和管理。

开发视图要考虑软件内部的需求，如软件开发的容易性、软件的重用和软件的通用性，要充分考虑由于具体开发工具的不同而带来的局限性。

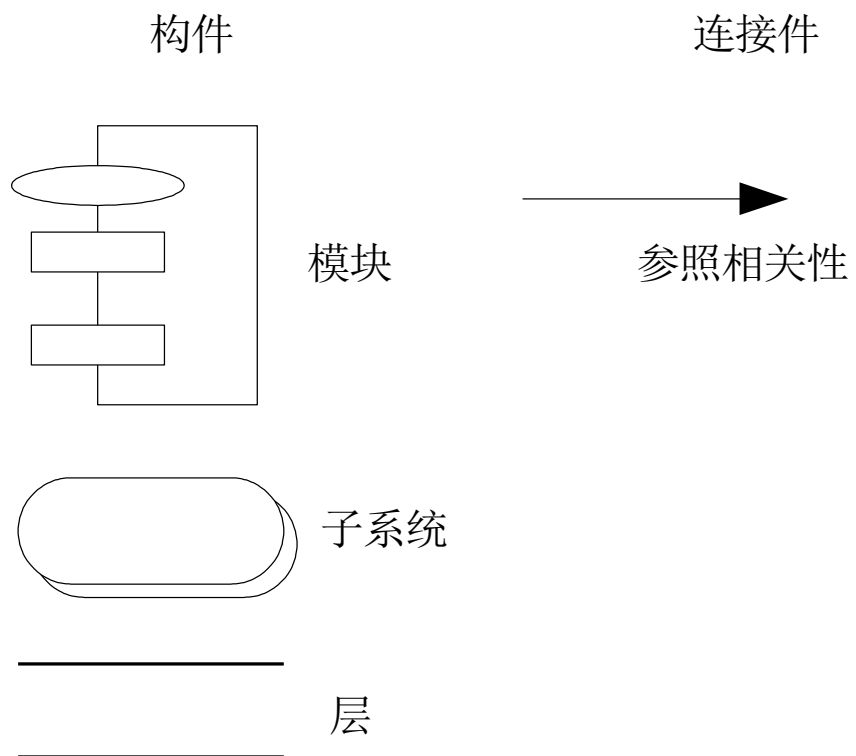
开发视图通过系统输入输出关系的模型图和子系统图来描述。

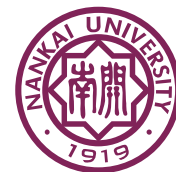
开发视图



南开大学
Nankai University

与逻辑视图一样，可以使用Booch标记法中某些符号来表示开发视图。



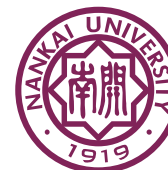


开发视图

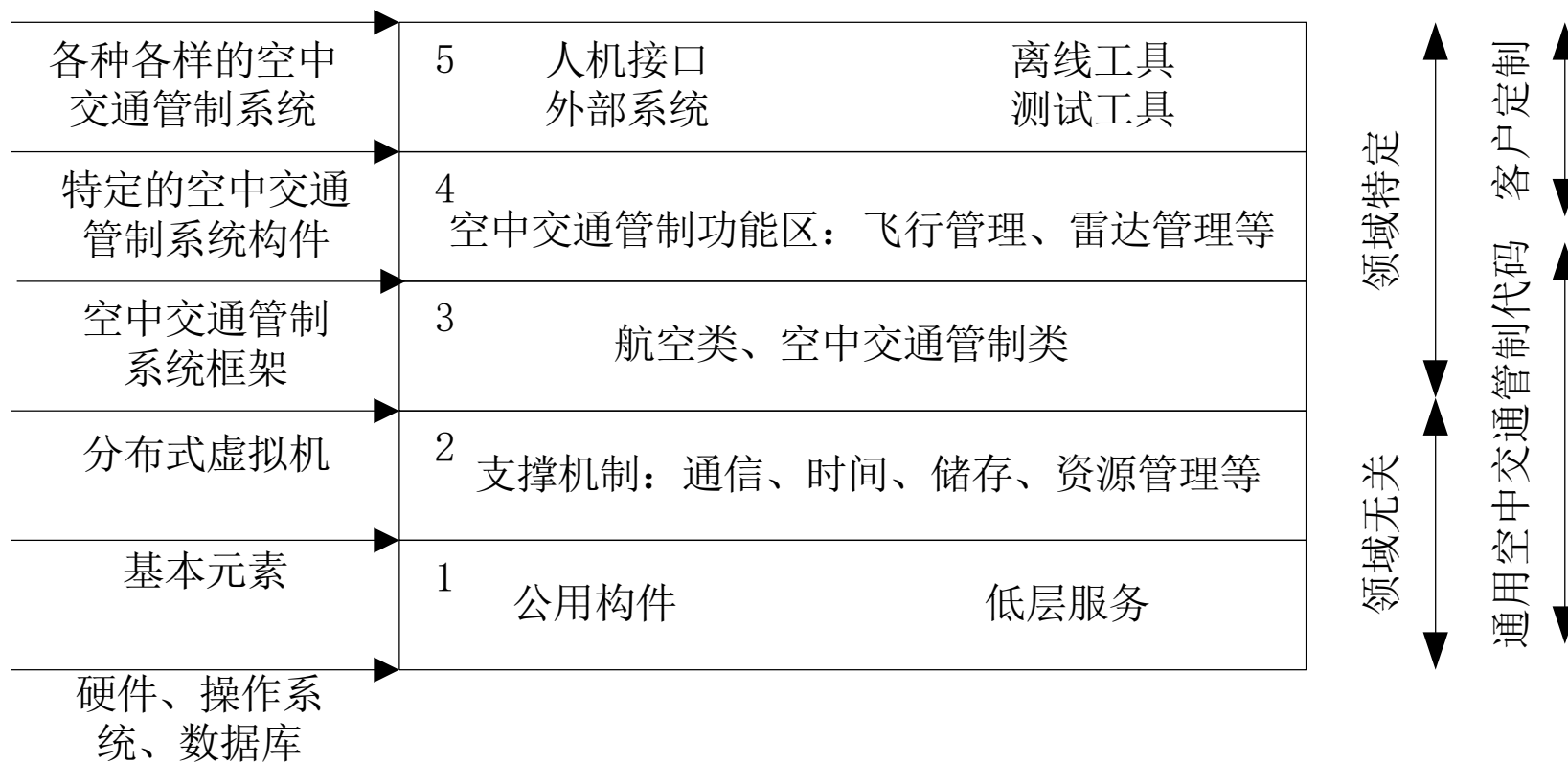
在开发视图中，最好采用4-6层子系统，而且每个子系统仅仅能与同层或更低层的子系统通讯，这样可以使每个层次的接口既完备又精练，避免了各个模块之间很复杂的依赖关系。

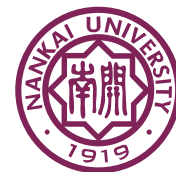
设计时要充分考虑，对于各个层次，层次越低，通用性越强，这样，可以保证应用程序的需求发生改变时，所做的改动最小。开发视图所用的风格通常是层次结构风格。

开发视图



南开大学
Nankai University



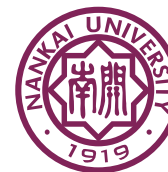


进程视图

进程视图侧重于系统的运行特性，主要关注一些非功能性的需求。

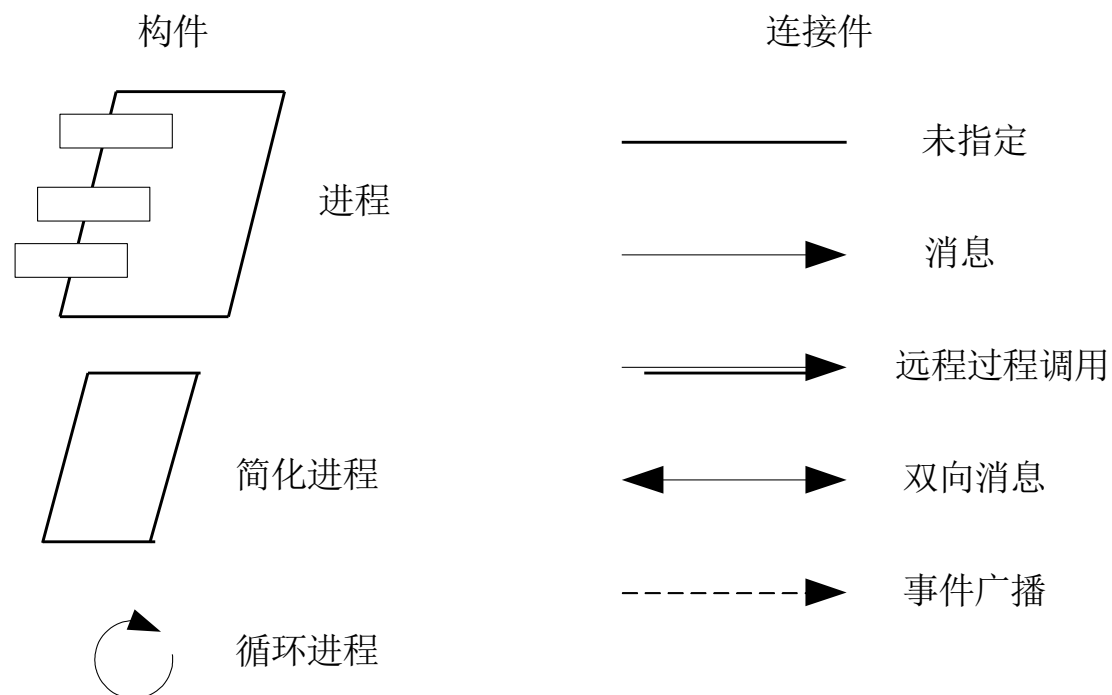
进程视图强调并发性、分布性、系统集成性和容错能力，以及从逻辑视图中的主要抽象如何适合进程结构。它也定义逻辑视图中的各个类的操作具体是在哪一个线程中被执行的。

进程视图可以描述成多层抽象，每个级别分别关注不同的方面。在最高层抽象中，进程结构可以看作是构成一个执行单元的一组任务。它可看成一系列独立的，通过逻辑网络相互通信的程序。它们是分布的，通过总线或局域网、广域网等硬件资源连接起来。

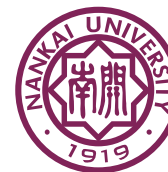


进程视图

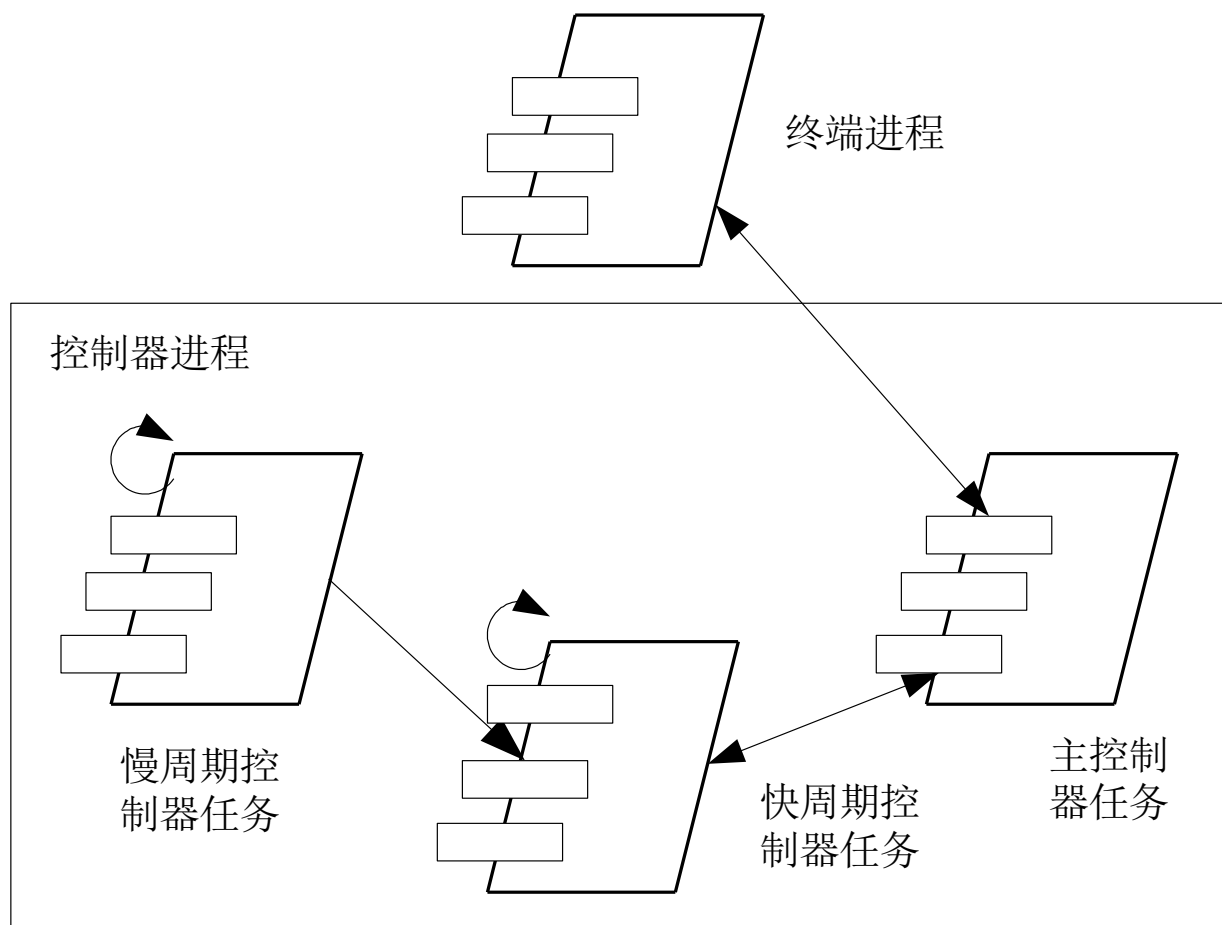
通过扩展Booch对Ada任务的表示法，来表示进程视图。

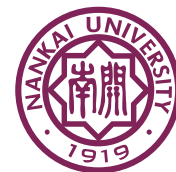


进程视图



南开大学
Nankai University

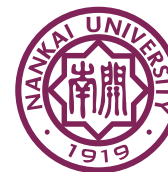




物理视图

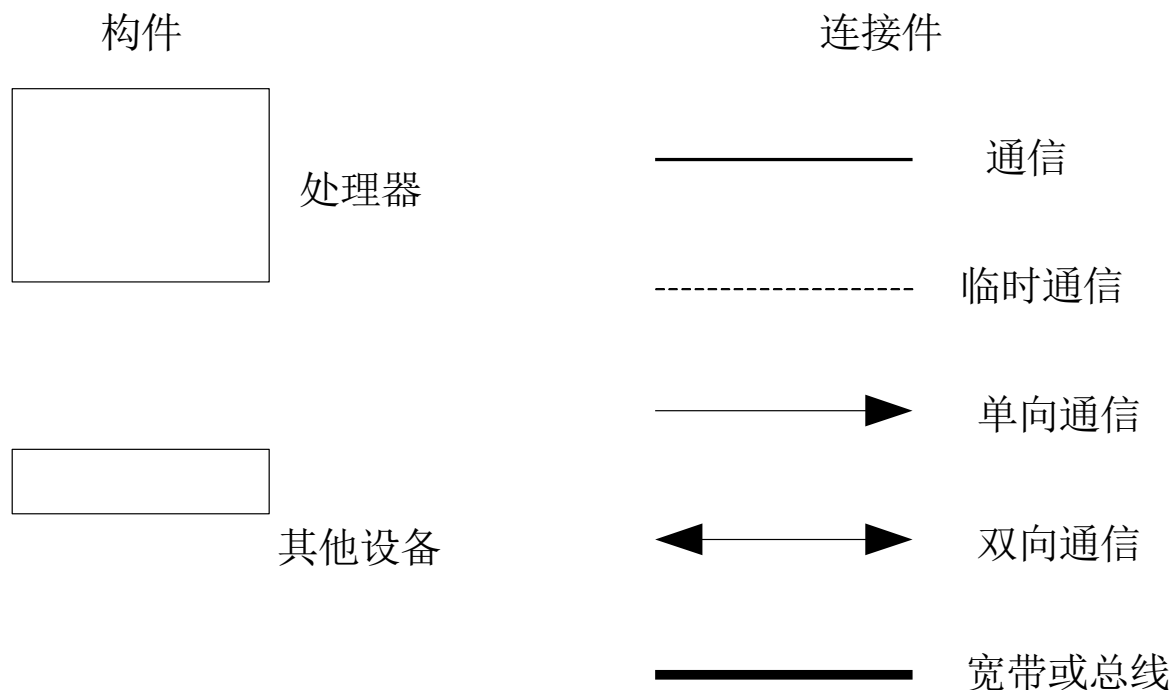
物理视图主要考虑如何把软件映射到硬件上，它通常要考虑到系统性能、规模、可靠性等。解决系统拓扑结构、系统安装、通讯等问题。

当软件运行于不同的节点上时，各视图中的构件都直接或间接地对应于系统的不同节点上。因此，从软件到节点的映射要有较高的灵活性，当环境改变时，对系统其他视图的影响最小。

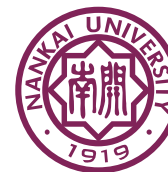


物理视图

大型系统的物理视图可能会变得十分混乱，因此可以与进程视图的映射一道，以多种形式出现，也可单独出现。

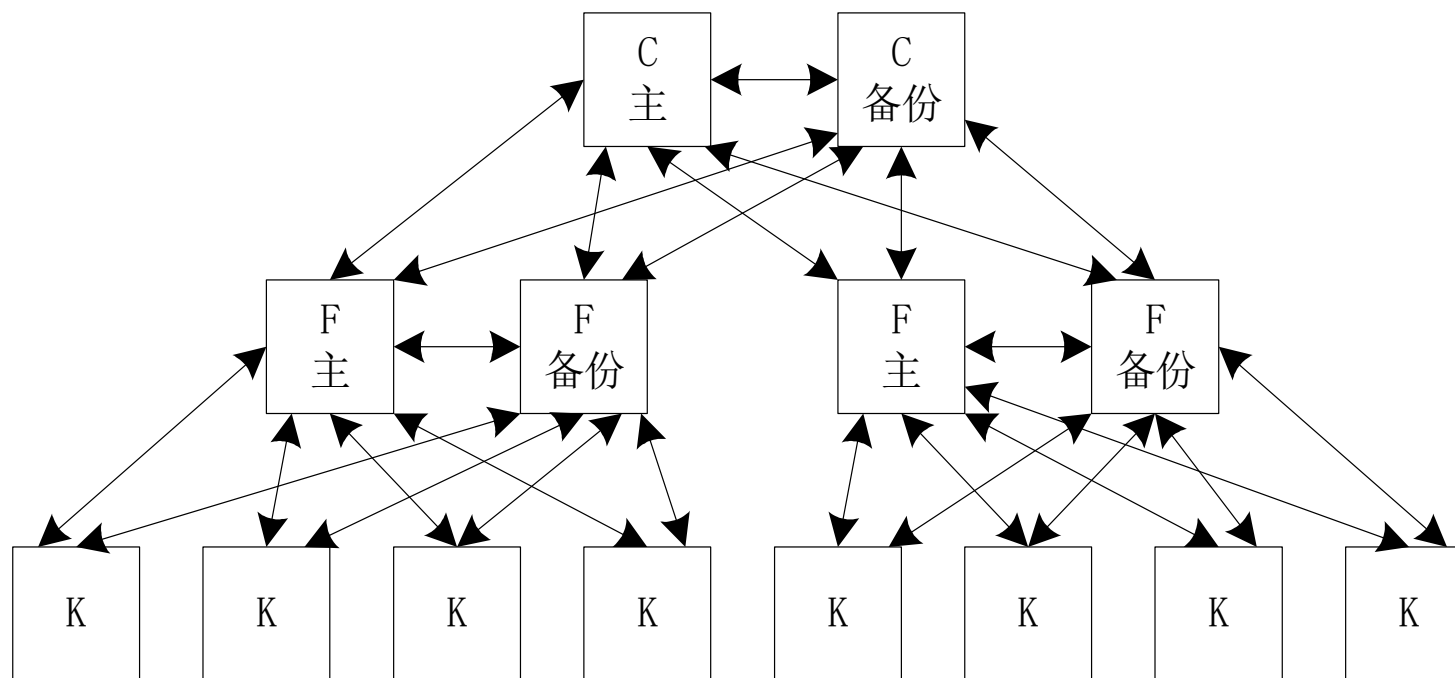


物理视图

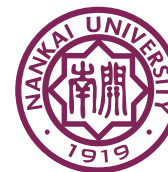


南开大学
Nankai University

ACS系统的物理视图

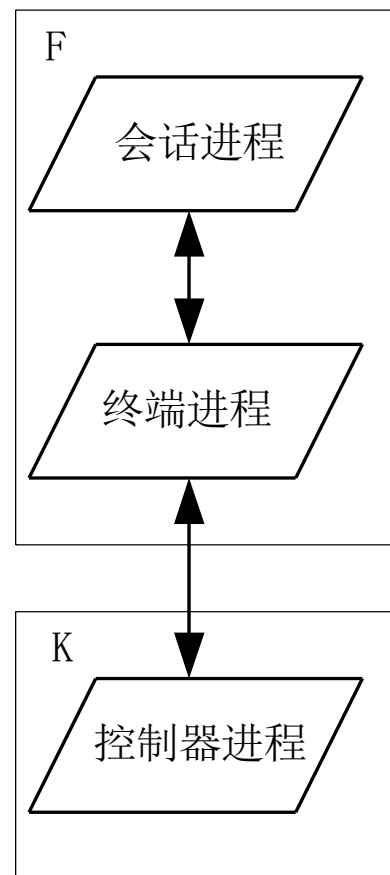


物理视图



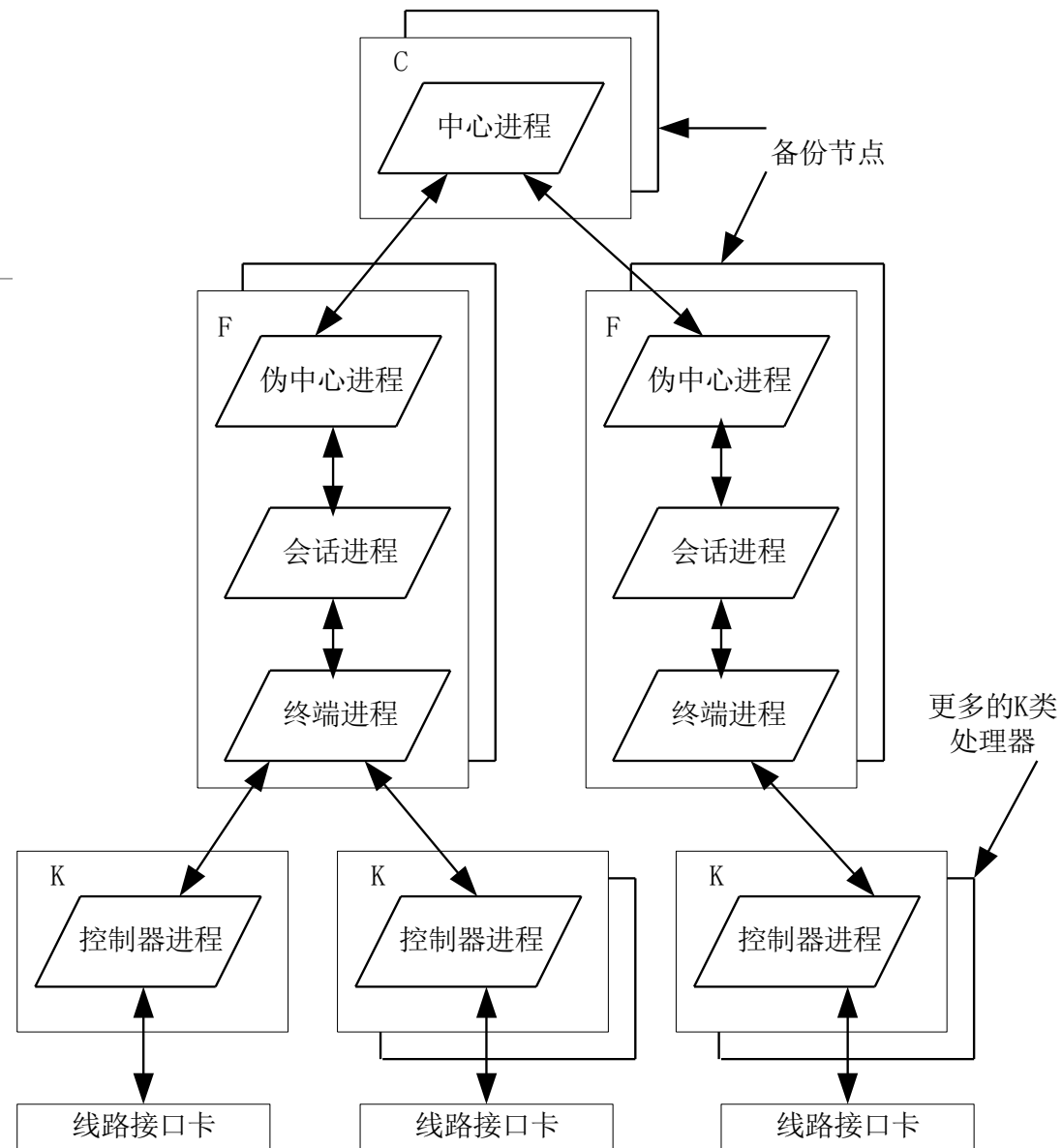
南开大学
Nankai University

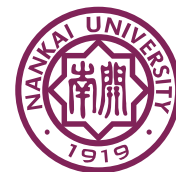
具有进程分配的小型ACS
系统的物理视图



物理视图

具有进程分配的大型ACS系统的物理视图



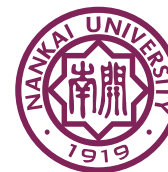


场景视图

场景可以看作是那些重要系统活动的抽象，它使四个视图有机联系起来，从某种意义上说场景是最重要的需求抽象。在开发体系结构时，它可以帮助设计者找到体系结构的构件和它们之间的作用关系。同时，也可以用场景来分析一个特定的视图，或描述不同视图构件间是如何相互作用的。

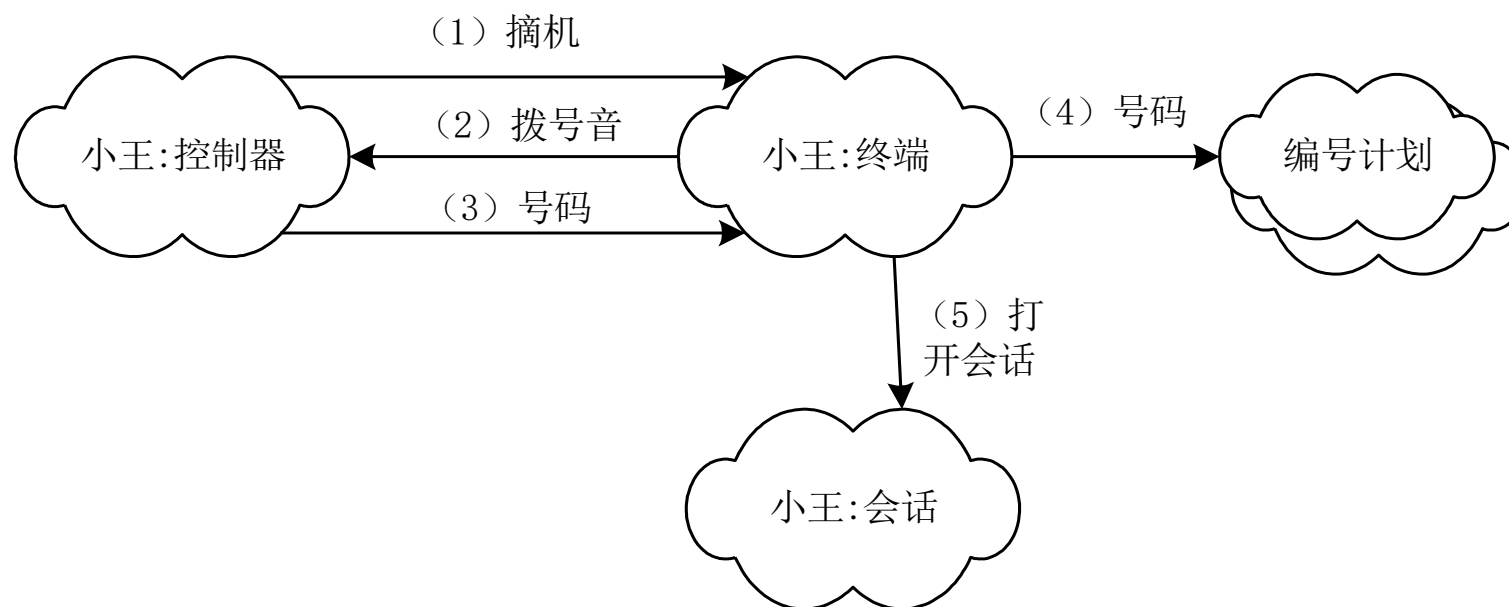
场景可以用文本表示，也可以用图形表示。

场景视图



南开大学
Nankai University

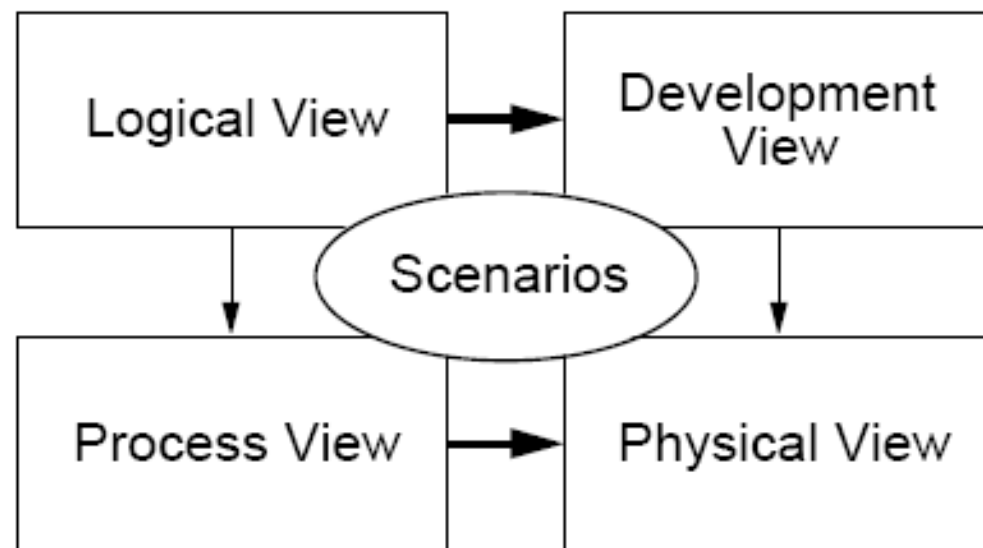
本地呼叫场景的一个原型





End-user
Functionality

Programmers
Software management



逻辑视图: 对象图, 动

进程视图: 活

开发视图: 状态图和活

物理视图: 题。部署图,

场景视图: 系统里要活动的抽象, 静态方面由用例图描述, 动态方面由协助图、状态图和活动图

静态方面由类图、图描述

组件图, 协作图、

题。部署图,

例子：电子商务系统的"4+1"视图模型

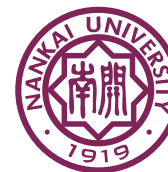
1. 逻辑视图（Logical View）

描述：关注系统的功能和业务逻辑，展示系统如何分解为不同的软件组件、类或者对象，以及它们之间的关系。

实例：

- 组件：用户模块、商品模块、购物车模块、订单模块、支付模块。
- 关系：用户可以与商品交互，将商品添加到购物车，从购物车生成订单，并通过支付模块完成支付。

例子



南开大学
Nankai University

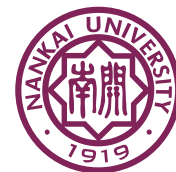
2. 开发视图（Development View）

描述：关注软件在开发环境下的模块组织，展示系统如何分解为不同的软件包、库、框架、工具等，以及它们之间的依赖关系。

实例：

- 软件包：用户接口包、业务逻辑包、数据访问包、第三方库包（如支付接口库）。
- 依赖关系：业务逻辑包依赖于数据访问包和第三方库包，用户接口包依赖于业务逻辑包。

例子



南开大学
Nankai University

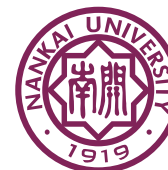
3. 进程视图（Process View）

描述：关注系统的运行时特性，展示系统如何分解为不同的进程和线程，以及它们之间的通信机制。

实例：

- 进程：Web服务器进程、应用服务器进程、数据库服务器进程。
- 通信：Web服务器进程通过HTTP与应用服务器进程通信，应用服务器进程通过JDBC与数据库服务器进程通信。

例子



南开大学
Nankai University

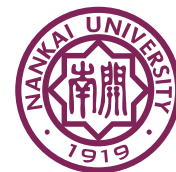
4. 物理视图（Physical View）

描述：关注系统在计算机环境中的物理分布，展示系统如何部署在不同的物理节点（如服务器、网络设备等）上，以及这些节点之间的连接关系。

实例：

- 节点：Web服务器节点、应用服务器节点、数据库服务器节点。
- 连接：Web服务器节点和应用服务器节点通过局域网连接，应用服务器节点和数据库服务器节点通过高速网络连接。

例子



南开大学
Nankai University

5. 场景视图（Scenarios）

描述：通过一系列的场景（如用例、用户故事等）来描述系统的行为，展示系统如何满足用户需求。

实例：

- 场景1：用户登录系统，浏览商品，将商品添加到购物车。
- 场景2：用户从购物车生成订单，并通过支付模块完成支付。
- 场景3：系统管理员管理商品信息，包括添加、修改和删除商品。

这些视图共同构成了系统的完整架构描述，帮助开发人员从不同的角度理解和设计系统。

例子：电子商务

1. 逻辑视图（Logical View）

描述：关注系统的功能和业务逻辑，展示系统如何分解为不同的软件组件、类或者对象，以及它们之间的关系。

实例：

- 组件：用户模块、商品模块、购物车模块、订单模块、支付模块。
- 关系：用户可以与商品交互，将商品添加到购物车，从购物车生成订单，并通过支付模块完成支付。

```
1  @startuml
2  package "电子商务系统" {
3      class User {
4          +login(credentials) : boolean
5          +browseProducts() : void
6          +addToCart(product) : void
7      }
8
9      class Product {
10         -name : string
11         -price : decimal
12         +getName() : string
13         +getPrice() : decimal
14     }
15
16     class Cart {
17         -products : Product[]
18         +addProduct(product) : void
19         +generateOrder() : Order
20     }
21
22     class Order {
23         -products : Product[]
24         -totalPrice : decimal
25         +placeOrder() : void
26     }
27
28     class Payment {
29         +processPayment(order) : boolean
30     }
31 }
32 @enduml
```

例子

2. 开发视图（Development View）

描述：关注软件在开发环境下的模块组织，展示系统如何分解为不同的软件包、库、框架、工具等，以及它们之间的依赖关系。

实例：

- 软件包：用户接口包、业务逻辑包、数据访问包、第三方库包（如支付接口库）。
- 依赖关系：业务逻辑包依赖于数据访问包和第三方库包，用户接口包依赖于业务逻辑包。

```
1  @startuml
2  package "电子商务系统" {
3      package "用户接口" {
4          class UserInterface
5      }
6
7      package "业务逻辑" {
8          class UserLogic
9          class ProductLogic
10         class CartLogic
11         class OrderLogic
12         class PaymentLogic
13     }
14
15     package "数据访问" {
16         class DatabaseAccess
17     }
18
19     package "第三方库" {
20         class ThirdPartyLibrary
21     }
22 }
23 @enduml
```

例子

3. 进程视图（Process View）

描述：关注系统的运行时特性，展示系统如何分解为不同的进程和线程，以及它们之间的通信机制。

实例：

- 进程：Web服务器进程、应用服务器进程、数据库服务器进程。
- 通信：Web服务器进程通过HTTP与应用服务器进程通信，应用服务器进程通过JDBC与数据库服务器进程通信。

```
1  @startuml
2  activity "用户购物流程" {
3      start
4      partition "Web服务器" {
5          User -> UserInterface: 登录
6          UserInterface -> UserLogic: 转发登录请求
7          UserLogic -> DatabaseAccess: 验证用户信息
8          DatabaseAccess --> UserLogic: 返回验证结果
9          UserLogic --> UserInterface: 返回登录结果
10
11         User -> UserInterface: 浏览商品
12         UserInterface -> ProductLogic: 转发浏览请求
13         ProductLogic -> DatabaseAccess: 获取商品信息
14         DatabaseAccess --> ProductLogic: 返回商品信息
15         ProductLogic --> UserInterface: 显示商品信息
16
17         User -> UserInterface: 添加到购物车
18         ...
19     }
20     ...
21 end
22 }
23 @enduml
```

例子

4. 物理视图（Physical View）

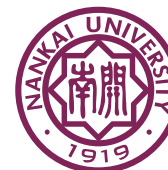
描述：关注系统在计算机环境中的物理分布，展示系统如何部署在不同的物理节点（如服务器、网络设备等）上，以及这些节点之间的连接关系。

实例：

- **节点：**Web服务器节点、应用服务器节点、数据库服务器节点。
- **连接：**Web服务器节点和应用服务器节点通过局域网连接，应用服务器节点和数据库服务器节点通过高速网络连接。

```
1  @startuml
2  deployment "电子商务系统部署" {
3      node "Web服务器" {
4          UserInterface
5      }
6
7      node "应用服务器" {
8          UserLogic
9          ProductLogic
10         CartLogic
11         OrderLogic
12         PaymentLogic
13     }
14
15     node "数据库服务器" {
16         DatabaseAccess
17     }
18
19     Web服务器 -> "HTTP" <..> 应用服务器
20     应用服务器 -> "JDBC" <..> 数据库服务器
21 }
22 @enduml
```

例子



南开大学
Nankai University

5. 场景视图 (Scenarios)

描述：通过一系列的场景（如用例、用户故事等）来描述系统的行为，展示系统如何满足用户需求。

实例：

- 场景1：**用户登录系统，浏览商品，将商品添加到购物车。

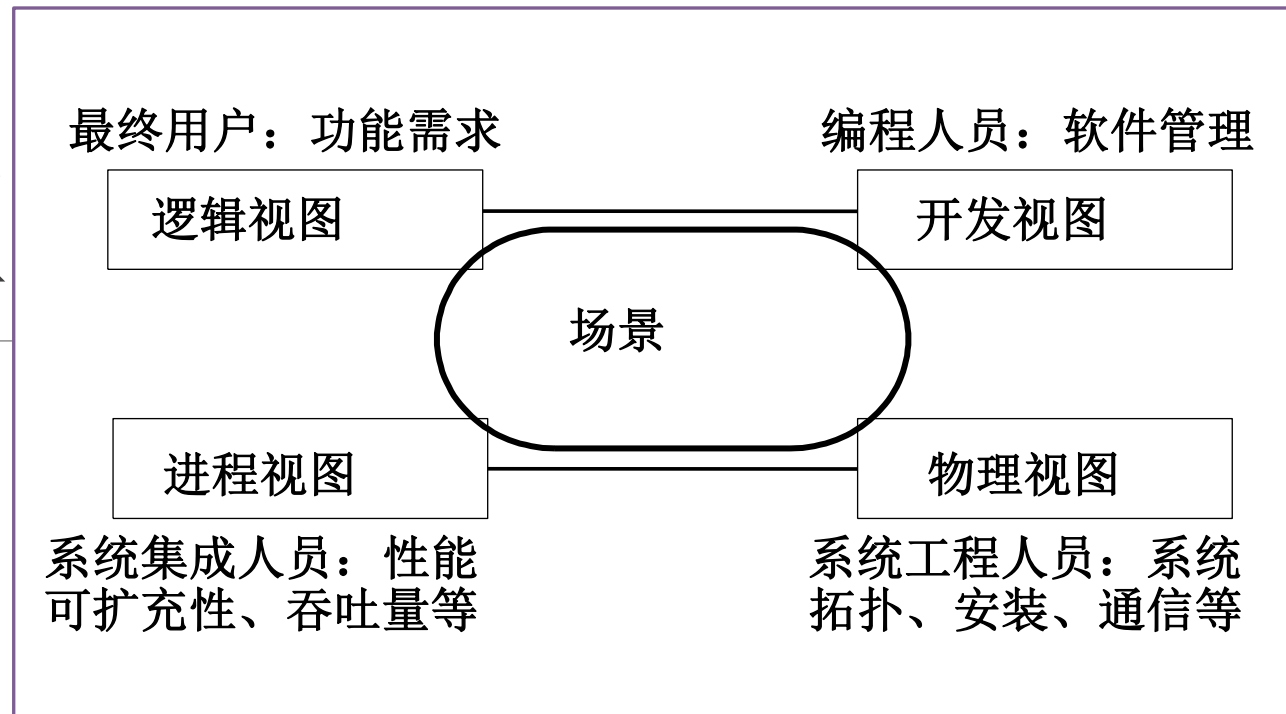
- 场景2：**用户从购物车生成订单，并通过支付模块完成支付。

- 场景3：**系统管理员管理商品信息，包括添加、修改和删除商品。

这些视图共同构成了系统的完整架构描述，帮助开发人员从不同的角度理解和设计系统。

```
1  @startuml
2  actor User
3  actor Admin
4
5  User -> (UserLogin)
6  User -> (BrowseProducts)
7  User -> (AddToCart)
8  User -> (Checkout)
9  User -> (Payment)
10
11 Admin -> (ManageProducts)
12
13 @enduml
```

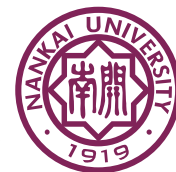
“4+1”的视图模型



逻辑视图和开发视图描述系统的静态结构，而进程视图和物理视图描述系统的动态结构。

对于不同的软件系统来说，侧重的角度也有所不同。例如，对于管理信息系统来说，比较侧重于从逻辑视图和开发视图来描述系统，而对于实时控制系统来说，则比较注重于从进程视图和物理视图来描述系统。

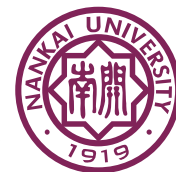
Rational 的“4+1”的视图模型



南开大学
Nankai University



Rational 的 “4+1”的视图模型



南开大学
Nankai University

不是所有系统都需要所有视图：

- 单一处理器：舍弃部署视图
- 单一进程：舍弃交互视图
- 小程序：舍弃实现视图

添加视图：

- 数据视图、安全视图。

Rational 的“4+1”的视图模型



南开大学
Nankai University

用例视图

- 用例视图包含描述用户、分析师和测试工程师看到的系统行为的用例。
- 根据视图可确定系统架构

UML:

- 静态方面由用例图描述
- 动态方面由交互图、状态图和活动图描述。

Rational 的“4+1”的视图模型



南开大学
Nankai University

设计视图

- 设计视图包含构建系统的类、接口和类之间的协作。
- 主要支持系统的功能性需求，也即系统提供给用户的服务。

UML:

- 静态方面由类图、对象图描述
- 动态方面由交互图、状态图和活动图描述。

Rational 的 “4+1”的视图模型



南开大学
Nankai University

交互视图

- 交互视图描述了系统不同部分之间的控制流，包括可能的并发和同步机制。
- 主要解决系统的性能、可拓展性、吞吐量等问题。

UML

- 静态方面和动态方面的描述所采用图都和设计视图相同。

Rational 的 “4+1”的视图模型



南开大学
Nankai University

实现视图

- 实现视图包含用于组装和发布物理系统的组件。
- 主要解决系统发布的配置管理问题。

UML:

- 静态方面用物件图描述。
- 动态方面用交互图、状态图和活动图描述。

Rational 的“4+1”的视图模型



南开大学
Nankai University

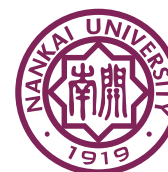
部署视图

- 部署视图包含形成系统硬件拓扑结构的节点。
- 主要解决构成物理系统的部件的分布、发布和安装问题。

UML:

- 静态方面由部署图描述
- 动态方面由交互图、状态图和活动图描述。

其它的视图模型



南开大学
Nankai University

Siemens四视图（Hofmeister, Nord, Soni, 2000年）

概念视图

互连视图

执行视图

代码视图

Herzum&Sims(BusinessComponentFactory,1999年)

技术架构

应用架构

项目管理架构

功能架构

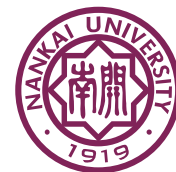
降低软件成本的方法（Parnas等人，1980年代）

模块分解视图：将模块显示为封装单位；用于隔离变化以及实现其可修改性

流程视图：显示流程及其在运行时如何进行同步和通信；用于实现性能

使用视图：显示程序以及它们之间相互依赖关系；用于实现增量开发和快速部署子集的能力

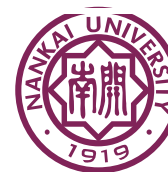
如何选择视图



南开大学
Nankai University

- 1 建立表格
- 2 适当组合视图以减少数量
- 3 根据 需求 对视图进行优先排序 某些利益相关者可能有额外的权重

体系结构的核心模型



南开大学
Nankai University

